# RISK LIST

| | |
|---|---|
| Spree | **5.6.1** |
| Stack | **Rails 8.1.3 · Ruby 4.0.1 · PostgreSQL 18** |
| Date | **13 September 2026** |

Written 11 September 2026, in answer to the question of whether this should be proposed to Spree as a standard feature or built as our own add-on.

Everything below marked as measured was measured against the running shops or read out of the installed gems on the same day.

---

## In one page

**The decision.** Build it as our own add-on. If it proves itself against real disputes, that working extension becomes the proposal to Spree. Those are not two options, they are the order.

**Build the dispute capture before the list.** Nothing in the stack currently knows a dispute has happened: no table, no payment state, and neither payment gem mentions the word. So today the list can only be filled by someone remembering, there is no history to start it from, and nobody is told a dispute has opened even though the deadline to respond is in the provider's own data.

**Three corrections to the design, each of which costs something if left alone.**

| | what happens if left as proposed |
|---|---|
| The IP field is empty on every order placed since changeover | Nothing writes it on the headless path, so it cannot match anything, and the dispute evidence field for the buyer's IP cannot be filled at all. Corrected 13 Sep: the CDN addresses in the first version were historical |
| The payment reference is unique per transaction | It can never match a future order, so the rule never fires and the system looks like it is working |
| Clearing a false match per customer or per order | The alert that should fire eighteen months later gets suppressed, or the wrong one is |

**The most likely harm.** On a stolen card, the cardholder is the victim, not the offender. Their name, billing address and card would go on the list, and under the cross-shop plan they would be refused at shops they have never used. The provider tells us which kind of dispute it is, so the record can treat those fields differently instead of as equally incriminating.

**The order to build it.** Capture disputes, then add records from them, then re-check open and unshipped orders the moment a record is added, then order-time alerts, then cross-shop last. The re-check is the step with the money in it, because a chargeback arrives a month or more after the order and the parcel can still be stopped.

The rest of this document is the evidence for each of those.

---

## The short answer

**Build it as our own add-on. Then, if it proves itself, the working extension is the proposal to Spree.**

Those are not two options, they are the order to do them in. Picking "keep it ours" alone reads as discouraging a sound idea. Picking "propose it to Spree" promises an outcome that other people control, and the 6.1 assistant is already an example of how long a roadmap item can sit with no code attached to it.

Three reasons it is not a core feature:

**It is a different axis from Spree's existing risk model, not an extension of it.** Spree has risk today, and it is this, in full:

```
def is_risky?              # spree_core-5.6.1 order.rb:923
  !payments.risky.empty?
end
```

A payment is risky when the gateway's AVS or CVV response says so. So Spree's risk is derived from the payment provider, computed per order at read time, with no memory of a person. `considered_risky` is a column, and in the whole gem it is written in exactly one place: `Orders::Approve` sets it to `false`. Spree never sets it to true.

The requirement here explicitly rejects provider-tied risk. So this is a second, unrelated axis that happens to share a word.

**Core structurally cannot do the half that is worth most.** The estate is eight separate Postgres databases across two servers. Spree's multi-store is several stores inside one database, which is a different thing. Nothing Spree ships to a single-database merchant can span eight databases.

**A list of named people held in order to refuse them service is a different category of record from an order.** Stripped of the interface it is a database of names, home addresses, telephone numbers and a free-text accusation of dishonesty. Shipping that to every merchant who upgrades is a liability Spree would reasonably not want to take on, and it would slow the feature down for good reasons.

---

## Six things the measurements changed

### 1. Nothing in the stack knows a dispute happened

| | **disputes mentioned** |
|---|---|
| `spree_core` 5.6.1 | no table, no payment state, no flag |
| `spree_stripe` 1.8.1 | **0 files** |
| `spree_paypal_checkout` 0.7.1 | **0 files** |

Payment states are `checkout, pending, completed, processing, failed, void, invalid, capture_pending, void_pending`. There is no disputed state and no dispute table. When the provider raises a dispute, nothing here hears it.

Two consequences. There is no history to start a list from: five completed orders across the estate carry a voided payment, and nothing distinguishes a chargeback from an ordinary void. And nobody is told a dispute has opened, which matters because the provider's dispute object carries `evidence_details.due_by`, a deadline to respond.

**So the first thing to build is not the risk list. It is capturing the dispute.** It is smaller, it is what feeds the list, and it is provider independent in exactly the intended sense: every provider emits a dispute event and the record stored is the same shape.

### 2. The reverse check is where the money is

A chargeback lands thirty to a hundred and twenty days after the order. A system that only matches orders as they arrive has an empty list on day one and stays a month behind for ever.

The direction with money in it is the opposite. The moment a risk record is added, re-check every open and unshipped order across the estate and surface the matches now. That is the case where a parcel can still be stopped. If the first version does one thing, it should be that, not order-time matching.

### 3. The IP field is empty, not wrong. Corrected 13 September

The first version of this section said the field was recording the CDN. That was measured across ALL orders, and it was the wrong measurement. Split by the changeover date instead:

| shop | before changeover | after changeover |
|---|---|---|
| A | 382 orders, **382** with an IP | 18 orders, **1** with an IP |
| B | 171 orders, **171** with an IP | 10 orders, **0** with an IP |

And the single exception is dated changeover day itself, carrying a CDN address, so it was placed on the old storefront before the switch finished.

So every CDN address in the first version was **historical**, written by the old Rails storefront. Since the shops went headless, nothing writes the field at all. A full column is not a written column; it can be full because of the past.

That changes what the work is. It is not a wrong value to correct, it is a path to create: the storefront has to pass the shopper's address on the API call that creates the order, and Spree has to record it. The web server has already been corrected to see the real address rather than the edge, which was necessary, but that is the precondition rather than the repair.

Either way the conclusion for the risk list is unchanged, and now for a blunter reason. IP cannot be a matching identifier today because there is nothing in the field. And the provider's dispute evidence form has a `customer_purchase_ip` field that cannot be filled at all, which is evidence not submitted in exactly the kind of case recently lost.

Drop IP from the list until the real client address is being recorded, and treat recording it as its own small piece of work.

**What the shops can already produce is worth saying, because it is most of a dispute response.** Measured on completed orders: tracking is present on 180 of 181 on one shop and 396 of 399 on another, and those values are genuine rather than placeholders, 179 and 393 of them distinct. Shipped dates are set on nearly all. That is proof of delivery, which is what wins a dispute claiming the goods never arrived.

Two gaps beside it. There is no carrier field on a shipment, only a tracking number, and the provider asks for the carrier by name; each shop uses a single postage method, so it is a per-shop constant rather than something to look up. And the billing address is missing on 49 of 181 completed orders on one shop, which is another field the evidence form asks for.

### 4. The payment identifier in the list is the wrong one

A payment identifier or reference is the transaction id. It is unique to that transaction, so it can never match a future order. The field would never fire, and a rule that never fires makes a system look like it is working.

The thing that matches is the instrument: the card fingerprint, which is stable across transactions for the same card. That is the single strongest identifier available, because name, email, telephone and address are all trivially changed and the card usually is not.

### 5. The most likely harm is flagging the victim

In a stolen card dispute the cardholder is not the offender. The name, the billing address and the card belong to someone who has already been a victim once. Adding those to a risk list means refusing them the next time they legitimately buy, and under the cross-shop plan, refusing them at shops they have never dealt with. It is also the case most likely to become a public complaint, because the flagged person is provably innocent.

The design consequence is concrete. The dispute reason is machine readable, and the provider's own enum separates `fraudulent` and `unrecognized` from `product_not_received` and `customer_initiated`. On a stolen card dispute the delivery address and the contact details are the offender's signal, while the name, billing address and card are the victim's. The record has to carry that distinction rather than treating every field as equally incriminating.

### 6. Clear as a false match must remember the pairing

Clearing per customer, or per order, are the two granularities that do not work.

The case that breaks them: an innocent person moves into a flagged delivery address and staff correctly clear the alert. Eighteen months later the original offender orders to that same address again on the same card. A clear stored against the address suppresses the alert that should fire. A clear stored against the customer suppresses nothing, because it is a different customer.

The unit to store is the triple: which risk record, which fields matched, and which identity they matched. That survives both cases.

---

## Two smaller points that change the build

**An alert with no hold is a note, not a control.** Eight shops taking orders at all hours means an alert that only paints a badge depends on somebody opening eight listings before anything ships. It needs to be able to stop the irreversible step. It should do that through the approval concept Spree already has, rather than a parallel one beside it, because hanging it on existing objects is what keeps it upgradeable and reusable rather than a fork.

**It must never sit in the customer's request path.** There is no version of this that needs to run while somebody is checking out, and putting it there turns a risk list outage into a checkout outage.

---

## Privacy, as engineering decisions rather than advice

The shops' own policies cite the Privacy Act 1988 and the Australian Privacy Principles. Three consequences follow without needing a lawyer:

- **The subject can ask to see what is held about them.** Free-text admin notes should therefore be written as facts with dates and order numbers, not opinions. That is a house rule and a placeholder in the form, not a technical control.
- **A record should expire or be reviewed**, rather than living for ever by default. Worth noting that a deleted record still exists in thirty days of database dumps, so deletion is not instant and should not be described as if it were.
- **Who can see it matters, and Spree's permissions are coarser than they look.** Already measured on this estate: one admin screen turned out to be gated by a broader permission than the obvious one, so a role meant only to view products could reach it. Whatever gates the risk list has to be checked by opening the screen as each role, not by reading the configuration and assuming.

---

## What the first version is

1. Record the dispute when the provider raises it. Store the provider, the reason, the status, the deadline and the order it belongs to.
2. From a dispute or an order, add a risk record, with the card fingerprint as the strongest field and the reason carried through.
3. On adding a record, re-check open and unshipped orders and surface matches now.
4. Order-time matching, alert with the ability to hold, and clear-as-false-match keyed on the pairing.
5. Cross-shop last, as a read across shops rather than a shared database, so that each record stays in the shop that created it and a partial answer is reported as partial rather than as clean.

A shared database is not merely the less tidy option here, it is closed off by how the estate is secured. Measured: Postgres runs with `trust` authentication, which means no password at all, and it is safe because no container port is published and every shop binds to loopback. The host exposes only SSH, 80 and 443. Reaching one

database from the other server would mean exposing a passwordless Postgres across the internet. So the cross-shop check has to be an authenticated read between shops, which is the same shape as the other cross-shop work already under way.

### Two smaller corrections to the design as written

**Severity should key on the strongest single identifier, not on how many matched.** Counting matches sounds right and is not: an offender who reuses one card while changing name, email, phone and address produces exactly one match, and that single match is near-certain identification. Meanwhile four weak matches on a shared household address are weak four times over. Rank by the best evidence present, not by the total.

**If it goes to Spree at all, send the hook rather than the feature.** The useful thing to ask upstream for is small and carries no personal data: an event when an order is placed and a place to hang an advisory on an order. That is a reasonable thing for Spree to accept, it is what the add-on would attach to, and it does not ask them to take on a database of named people.

Steps one to four are a single-shop add-on, which is the part general enough to be worth offering to Spree once it has been run against real disputes. Step five is the part that is ours, and it is the part with the most value in this estate.